CGE-P LAB CURRICULUM · V2

Lab 3.3: Writing Compliance Policies in Rego (GCP)

You wrote Terraform that satisfies controls. Now you write the policy that proves a plan satisfies them before it ever applies. Five policies, five controls, one library that survives a cloud change.

The `# METADATA` blocks are not decoration. Lab 6.1 extracts them to build the OSCAL component, so each control mapping is authored once and consumed twice.

For reading. Code lines wrap to fit the page; copy code from docs/03_03_writing_compliance_policies_rego.md, not the PDF.

GRC Engineering Club

grcengclub.com · based on GRCEngClub/cgep-labs

FROM CHECKBOX GRC TO ENGINEERED SYSTEMS

This PDF is for reading. Long code lines wrap to fit the page, so copy commands and code from `docs/03_03_writing_compliance_policies_rego.md` in your clone, not from the PDF.

Learning objectives

- Author Rego policies with structured annotations that map each rule to a NIST control.
- Write `_test.rego` fixtures asserting both passing and failing behavior.
- **Mutation-test every policy**, so you know it can fail, not merely that it passed.
- Extract policy metadata programmatically with `opa inspect`, so the mapping feeds Chapter 6 instead of decorating a comment.

The five policies

Control	File	Enforces
SC-28	`sc28_encryption.rego`	Every `google_storage_bucket` has a CMEK `encryption` block.
AC-3	`ac3_no_public.rego`	Uniform access + `public_access_prevention = "enforced"`; firewalls do not expose 22 or 3389 to `0.0.0.0/0`.
CM-6	`cm6_required_tags.rego`	Four required labels on every taggable resource.
AU-3	`au3_access_logging.rego`	**New in v2.** Every bucket has a `logging` block naming a target.
AU-9	`au9_log_immutability.rego`	**New in v2.** The bucket named as a logging target is itself versioned.

Every deny message carries the resource address **and** the control ID. The developer fixes their own violation without filing a GRC ticket.

Prerequisites

- Run these from inside the devcontainer if you set one up in Lab 0.1: that is where the toolchain and your cloud logins live. `source cgep.env` first, in every new shell.
- OPA `>= 1.0` (tested on 1.19.1). Policies declare `import rego.v1`, so they also run on 0.x `>= 0.60`.
- Terraform `>= 1.9` and the `google` provider authenticated (`gcloud auth application-default login`).
- Lab 2.4 completed. Its module is what the fixture consumes, and its `compliance_attestation` is what the AU-9 rule leans on.

Estimated time & cost

- 75 to 90 minutes. The mutation-testing section accounts for much of that, and it is the part worth the time.
- Free. Everything runs against `terraform plan` output. Nothing is applied.

Step-by-step walkthrough

Step 1 Structure

```
mkdir -p policies/tests terraform fixtures
```

Step 2 The mixed fixture

A fixture with compliant and non-compliant resources gives the suite something concrete to flag. This is Lab 2.4's world, deliberately broken in five specific ways.

```
# terraform/main.tf
terraform {
  required_version = ">= 1.9"
  required_providers {
    google = { source = "hashicorp/google", version = "~> 5.0" }
  }
}

provider "google" {
  project = var.gcp_project
  region  = "us-central1"
}

variable "gcp_project" { type = string }

locals {
  labels = {
    project          = "lab33"
    environment      = "dev"
    managed_by       = "terraform"
    compliance_scope = "cge-p-lab"
  }
}

resource "google_kms_key_ring" "ring" {
  name     = "lab33-ring"
  location = "us-central1"
}

resource "google_kms_crypto_key" "key" {
```

```
  name     = "lab33-key"
  key_ring = google_kms_key_ring.ring.id
}

# A compliant log bucket: versioned, so AU-9 is satisfiable.
resource "google_storage_bucket" "logs" {
  name                        = "${var.gcp_project}-lab33-logs"
  location                    = "us-central1"
  uniform_bucket_level_access = true
  public_access_prevention    = "enforced"

  versioning { enabled = true }

  encryption { default_kms_key_name = google_kms_crypto_key.key.id }

  labels = local.labels
}

# An UNVERSIONED log bucket. AU-9 should fire on anything logging here.
resource "google_storage_bucket" "bad_logs" {
  name                        = "${var.gcp_project}-lab33-badlogs"
  location                    = "us-central1"
  uniform_bucket_level_access = true
  public_access_prevention    = "enforced"

  encryption { default_kms_key_name = google_kms_crypto_key.key.id }

  labels = local.labels
}

# Fully compliant.
resource "google_storage_bucket" "good" {
  name                        = "${var.gcp_project}-lab33-good"
  location                    = "us-central1"
  uniform_bucket_level_access = true
  public_access_prevention    = "enforced"

  versioning { enabled = true }
  encryption { default_kms_key_name = google_kms_crypto_key.key.id }
  logging {
    log_bucket        = google_storage_bucket.logs.name
    log_object_prefix = "access-logs/"
  }

  labels = local.labels
}

# SC-28 should fire: no encryption block.
resource "google_storage_bucket" "bad_no_cmek" {
  name                        = "${var.gcp_project}-lab33-no-cmek"
  location                    = "us-central1"
  uniform_bucket_level_access = true
  public_access_prevention    = "enforced"

  logging { log_bucket = google_storage_bucket.logs.name }
  labels  = local.labels
```

```
}

# AC-3 should fire: uniform access off, prevention not enforced.
resource "google_storage_bucket" "bad_public" {
  name                        = "${var.gcp_project}-lab33-public"
  location                    = "us-central1"
  uniform_bucket_level_access = false
  public_access_prevention    = "inherited"

  encryption { default_kms_key_name = google_kms_crypto_key.key.id }
  logging    { log_bucket = google_storage_bucket.logs.name }
  labels     = local.labels
}

# CM-6 should fire: no labels.
resource "google_storage_bucket" "bad_no_labels" {
  name                        = "${var.gcp_project}-lab33-no-labels"
  location                    = "us-central1"
  uniform_bucket_level_access = true
  public_access_prevention    = "enforced"

  encryption { default_kms_key_name = google_kms_crypto_key.key.id }
  logging    { log_bucket = google_storage_bucket.logs.name }
}

# AU-3 should fire: no logging block at all.
resource "google_storage_bucket" "bad_no_logging" {
  name                        = "${var.gcp_project}-lab33-no-logging"
  location                    = "us-central1"
  uniform_bucket_level_access = true
  public_access_prevention    = "enforced"

  encryption { default_kms_key_name = google_kms_crypto_key.key.id }
  labels     = local.labels
}

# AU-9 should fire: logs to a bucket that is not versioned.
resource "google_storage_bucket" "bad_log_target" {
  name                        = "${var.gcp_project}-lab33-badtarget"
  location                    = "us-central1"
  uniform_bucket_level_access = true
  public_access_prevention    = "enforced"

  encryption { default_kms_key_name = google_kms_crypto_key.key.id }
  logging    { log_bucket = google_storage_bucket.bad_logs.name }
  labels     = local.labels
}

resource "google_compute_network" "demo" {
  name                    = "lab33-demo"
  auto_create_subnetworks = false
}

# AC-3 should fire.
resource "google_compute_firewall" "open_ssh" {
  name          = "lab33-open-ssh"
```

```
  network       = google_compute_network.demo.name
  direction     = "INGRESS"
  source_ranges = ["0.0.0.0/0"]

  allow {
    protocol = "tcp"
    ports    = ["22"]
  }
}
```

```
cd terraform
terraform init
terraform plan -out=tfplan -var=gcp_project=your-gcp-project
terraform show -json tfplan > plan.json
```

You never apply. The policies operate on `plan.json`.

Step 3 SC-28

```
# policies/sc28_encryption.rego
# METADATA
# title: SC-28 - Encryption at Rest (GCS)
# description: "Every google_storage_bucket must encrypt at rest with a customer-managed encryption
key (CMEK)."
# authors:
#   - CGE-P
# custom:
#   control_id: SC-28
#   framework: nist-800-53-rev5
#   severity: high
#   remediation: "Add an encryption { default_kms_key_name = ... } block referencing a
google_kms_crypto_key you control."
package compliance.sc28

import rego.v1

deny contains msg if {
    some resource in all_buckets
    not has_cmek(resource)
    msg := sprintf(
        "[SC-28] %s: missing customer-managed encryption key. Remediation: add encryption
{ default_kms_key_name = ... }.",
        [resource.address],
    )
}

all_buckets contains r if {
    some r in input.planned_values.root_module.resources
    r.type == "google_storage_bucket"
}
```

```
all_buckets contains r if {
    some child in input.planned_values.root_module.child_modules
    some r in child.resources
    r.type == "google_storage_bucket"
}

has_cmek(resource) if {
    count(resource.values.encryption) > 0
    not empty_kms_key(resource.values.encryption[0])
}

empty_kms_key(enc) if enc.default_kms_key_name == ""
empty_kms_key(enc) if enc.default_kms_key_name == null
```

Why `has_cmek` checks the block, not the value. At plan time the KMS key ID is "(known after apply)" because the key does not exist yet, and the plan JSON omits unknown values entirely. Requiring a populated string would fail every compliant configuration. Accept a non-empty block; fail only when it is missing or explicitly empty. This is correct policy semantics, not a workaround.

Note the `all_buckets` helper. The obvious alternative is to duplicate the whole `deny` rule to recurse into `child_modules`, once per policy. Factoring the traversal into one set comprehension halves the length of every rule, and means a bug in the recursion is fixed in one place.

Step 4 AU-3 and AU-9, the two new ones

```
# policies/au3_access_logging.rego
# METADATA
# title: AU-3 - Content of Audit Records (GCS access logging)
# description: "Every google_storage_bucket must emit access logs to a named target bucket."
# custom:
#   control_id: AU-3
#   framework: nist-800-53-rev5
#   severity: medium
#   remediation: "Add a logging { log_bucket = ... } block naming a dedicated log bucket."
package compliance.au3

import rego.v1

deny contains msg if {
    some resource in buckets
    not is_log_sink(resource)
    not has_logging(resource)
    msg := sprintf(
        "[AU-3] %s: no access logging configured. Remediation: add logging { log_bucket = ... }.",
        [resource.address],
    )
}
```

```
buckets contains r if {
    some r in input.planned_values.root_module.resources
    r.type == "google_storage_bucket"
}

buckets contains r if {
    some child in input.planned_values.root_module.child_modules
    some r in child.resources
    r.type == "google_storage_bucket"
}

has_logging(resource) if {
    count(resource.values.logging) > 0
    resource.values.logging[0].log_bucket != ""
}

# A bucket that is itself a logging target is exempt. Without this exemption
# the rule demands infinite regress: every log bucket needs a log bucket.
is_log_sink(resource) if {
    some other in buckets
    count(other.values.logging) > 0
    other.values.logging[0].log_bucket == resource.values.name
}
```

That exemption is the interesting part, and it is the kind of thing you only discover by running the policy against a real plan. A naive AU-3 rule flags your log bucket for not having a log bucket, forever. Deciding where the recursion stops is a policy decision, and stating it in the rule is better than suppressing the finding later.

```
# policies/au9_log_immutability.rego
# METADATA
# title: AU-9 - Protection of Audit Information (log target versioning)
# description: "A bucket named as a logging target must have versioning enabled so audit records
cannot be silently overwritten."
# custom:
#   control_id: AU-9
#   framework: nist-800-53-rev5
#   severity: high
#   remediation: "Enable versioning { enabled = true } on the bucket used as a logging target."
package compliance.au9

import rego.v1

deny contains msg if {
    some target_name in log_target_names
    some bucket in buckets
    bucket.values.name == target_name
    not versioned(bucket)
    msg := sprintf(
        "[AU-9] %s: used as a logging target but versioning is disabled. Audit records can be silently
overwritten. Remediation: versioning { enabled = true }.",
        [bucket.address],
    )
}
```

```
buckets contains r if {
    some r in input.planned_values.root_module.resources
    r.type == "google_storage_bucket"
}

buckets contains r if {
    some child in input.planned_values.root_module.child_modules
    some r in child.resources
    r.type == "google_storage_bucket"
}

log_target_names contains name if {
    some b in buckets
    count(b.values.logging) > 0
    name := b.values.logging[0].log_bucket
}

versioned(bucket) if {
    count(bucket.values.versioning) > 0
    bucket.values.versioning[0].enabled == true
}
```

Write the rule that catches your own past mistake. It is the most reliable source of good policy ideas you will ever have, and this one catches a mistake almost everybody makes once: versioning the bucket that holds the data and forgetting the one that holds the record of who touched it.

Step 5 AC-3 and CM-6

Both use the shared `buckets` helper rather than repeating the traversal.

```
# policies/ac3_no_public.rego
# METADATA
# title: AC-3 - Access Enforcement (no public GCS, no open management ports)
# description: "GCS buckets must enforce uniform_bucket_level_access AND
public_access_prevention=enforced. Firewalls must not allow 0.0.0.0/0 on ports 22 or 3389."
# custom:
#   control_id: AC-3
#   framework: nist-800-53-rev5
#   severity: critical
#   remediation: "Set uniform_bucket_level_access = true and public_access_prevention = \"enforced\".
For firewalls, narrow source_ranges."
package compliance.ac3

import rego.v1

deny contains msg if {
    some r in buckets
    not locked_down(r)
    msg := sprintf(
        "[AC-3] %s: bucket allows public access. Remediation: set uniform_bucket_level_access=true and
public_access_prevention=\"enforced\".",
        [r.address],
```

```
    )
}

deny contains msg if {
    some r in input.planned_values.root_module.resources
    r.type == "google_compute_firewall"
    r.values.direction == "INGRESS"
    some src in r.values.source_ranges
    public_range(src)
    some allow in r.values.allow
    some port in allow.ports
    mgmt_port(port)
    msg := sprintf(
        "[AC-3] %s: management port %s open to %s. Remediation: narrow source_ranges or remove the
rule.",
        [r.address, port, src],
    )
}

buckets contains r if {
    some r in input.planned_values.root_module.resources
    r.type == "google_storage_bucket"
}

buckets contains r if {
    some child in input.planned_values.root_module.child_modules
    some r in child.resources
    r.type == "google_storage_bucket"
}

locked_down(r) if {
    r.values.uniform_bucket_level_access == true
    r.values.public_access_prevention == "enforced"
}

mgmt_port(p) if p == "22"
mgmt_port(p) if p == "3389"

public_range(s) if s == "0.0.0.0/0"
public_range(s) if s == "*"
```

```
# policies/cm6_required_tags.rego
# METADATA
# title: CM-6 - Configuration Settings (required compliance labels)
# description: "Every taggable resource must carry the four required labels."
# custom:
#   control_id: CM-6
#   framework: nist-800-53-rev5
#   severity: medium
#   remediation: "Add the four required labels, or consume the Lab 2.4 module which merges them."
package compliance.cm6

import rego.v1
```

```
required := {"project", "environment", "managed_by", "compliance_scope"}

labelable_type(t) if t == "google_storage_bucket"
labelable_type(t) if t == "google_compute_instance"
labelable_type(t) if t == "google_compute_disk"

deny contains msg if {
    some resource in all_resources
    labelable_type(resource.type)
    missing := required - provided_labels(resource)
    count(missing) > 0
    msg := sprintf(
        "[CM-6] %s: missing required labels %v. Remediation: add them, or consume the compliant
module.",
        [resource.address, sort_array(missing)],
    )
}

all_resources contains r if { some r in input.planned_values.root_module.resources }

all_resources contains r if {
    some child in input.planned_values.root_module.child_modules
    some r in child.resources
}

provided_labels(resource) := keys if {
    resource.values.labels
    keys := {k | resource.values.labels[k]}
}

provided_labels(resource) := set() if not resource.values.labels

sort_array(s) := sort([x | some x in s])
```

Set subtraction requires sets on both sides. `required - provided` fails if `provided_labels` returns an array. That is exactly why it is a set comprehension `{k | ...}` and not a list.

Step 6 Tests

One passing and one failing fixture per policy, minimum.

```
# policies/tests/au9_log_immutability_test.rego
package compliance.au9_test

import rego.v1
import data.compliance.au9

compliant := {"planned_values": {"root_module": {"resources": [
    {
        "address": "google_storage_bucket.app",
```

```
        "type": "google_storage_bucket",
        "values": {"name": "app", "logging": [{"log_bucket": "logs"}], "versioning": []},
    },
    {
        "address": "google_storage_bucket.logs",
        "type": "google_storage_bucket",
        "values": {"name": "logs", "logging": [], "versioning": [{"enabled": true}]},
    },
]}}}

noncompliant := {"planned_values": {"root_module": {"resources": [
    {
        "address": "google_storage_bucket.app",
        "type": "google_storage_bucket",
        "values": {"name": "app", "logging": [{"log_bucket": "logs"}], "versioning": []},
    },
    {
        "address": "google_storage_bucket.logs",
        "type": "google_storage_bucket",
        "values": {"name": "logs", "logging": [], "versioning": [{"enabled": false}]},
    },
]}}}

test_versioned_log_target_passes if {
    count(au9.deny) == 0 with input as compliant
}

test_unversioned_log_target_fails if {
    some msg in au9.deny with input as noncompliant
    contains(msg, "AU-9")
    contains(msg, "google_storage_bucket.logs")
}
```

Write the equivalent for the other four. Then:

```
opa test -v policies/
```

Step 7 Mutation testing, or: prove the policy can fail

This section is new in v2 and it is the most important part of the lab.

A passing test suite proves your policy returns the answer you expected on the inputs you thought of. It does not prove the policy is *load-bearing*. A rule with a typo in the resource type matches nothing, denies nothing, and passes every "compliant input produces zero denials" test you wrote. It is a control that cannot fail, which is to say it is not a control.

The check is mechanical: break the policy on purpose and confirm the tests notice.

```
#!/usr/bin/env bash
# scripts/mutation-test.sh
# For each policy, apply a mutation that should break it, and require that
# `opa test` FAILS. A mutation that survives means the tests do not constrain
# that rule, and the rule is decorative.
set -uo pipefail

FAILED=0
for f in policies/*.rego; do
  cp "$f" "$f.bak"

  # Mutation: invert every equality comparison in the rule body.
  sed -i 's/ == / != /g' "$f"

  if opa test policies/ >/dev/null 2>&1; then
    echo "SURVIVED: $f  -- tests still pass with the logic inverted."
    echo "           This rule is not constrained by any test."
    FAILED=1
  else
    echo "killed:   $f"
  fi

  mv "$f.bak" "$f"
done

exit $FAILED
```

```
chmod +x scripts/mutation-test.sh
bash scripts/mutation-test.sh
```

Every policy must report `killed`. A `SURVIVED` line means that policy has no test which actually depends on its logic, and you should assume it is not enforcing anything until you have written one that does.

This is worth doing once by hand to feel it. Comment out the body of your SC-28 `has_cmek` rule so it always returns true, run `opa test`, and watch which tests go red. If none do, your fixtures are not exercising the rule.

The general principle transfers well beyond Rego: **every compliance check must be mutation-tested before it is trusted.** An unfired control and an absent control look identical in a green pipeline.

Step 8 Metadata as data, not decoration

It is tempting to treat `# METADATA` as a comment. OPA parses it as structured annotations, and you can extract them:

```
opa inspect --annotations --format json policies/ \
  | jq '[.annotations[] | {
      control: .annotations.custom.control_id,
      severity: .annotations.custom.severity,
      title: .annotations.title,
      package: (.path | map(.value) | join(".")),
```

```
      remediation: .annotations.custom.remediation
    }]' > "$EVIDENCE/policy-catalog.json"

cat "$EVIDENCE/policy-catalog.json"
```

```
[
  { "control": "SC-28", "severity": "high",     "package": "data.compliance.sc28", "...": "..." },
  { "control": "AC-3",  "severity": "critical", "package": "data.compliance.ac3",  "...": "..." },
  { "control": "CM-6",  "severity": "medium",   "package": "data.compliance.cm6",  "...": "..." },
  { "control": "AU-3",  "severity": "medium",   "package": "data.compliance.au3",  "...": "..." },
  { "control": "AU-9",  "severity": "high",     "package": "data.compliance.au9",  "...": "..." }
]
```

That file is the bridge to Chapter 6. Lab 6.1 reads it to generate the `implemented-requirement` entries in your OSCAL component, so **the control mapping is authored once, in the policy that enforces it, and never retyped.** Every place a mapping is retyped is a place it can drift.

Step 9 Run the library against the real plan

```
opa test -v policies/
bash scripts/mutation-test.sh

for ns in sc28 ac3 cm6 au3 au9; do
  echo "=== $ns ==="
  opa eval -d policies -i terraform/plan.json "data.compliance.$ns.deny" --format=pretty
done
```

Expected, abridged:

```
=== sc28 ===
["[SC-28] google_storage_bucket.bad_no_cmek: missing customer-managed encryption key. ..."]

=== ac3 ===
["[AC-3] google_compute_firewall.open_ssh: management port 22 open to 0.0.0.0/0. ...",
 "[AC-3] google_storage_bucket.bad_public: bucket allows public access. ..."]

=== cm6 ===
["[CM-6] google_storage_bucket.bad_no_labels: missing required labels [...]. ..."]

=== au3 ===
["[AU-3] google_storage_bucket.bad_no_logging: no access logging configured. ..."]

=== au9 ===
["[AU-9] google_storage_bucket.bad_logs: used as a logging target but versioning is disabled. ..."]
```

Each broken resource flagged exactly once by the right control. The good bucket is quiet, and so is the compliant log bucket, because the AU-3 exemption works.

Verification

- `opa test -v policies/` passes, at least two tests per policy.
- `scripts/mutation-test.sh` reports `killed` for every policy and exits 0.
- Each `deny` set against `plan.json` contains exactly the expected violations.
- `evidence/lab-3-3/policy-catalog.json` lists all five controls.
- After fixing the fixture, every deny set is empty.

Capture the evidence the checklist asks for

```
# evidence/ lives at the repository root, not in the workspace you are in
EVIDENCE="$(git rev-parse --show-toplevel)/evidence/lab-3-3"
mkdir -p "$EVIDENCE"
opa test policies/ --format=json > "$EVIDENCE/opa-test-results.json"
bash scripts/mutation-test.sh 2>&1 | tee "$EVIDENCE/mutation-results.txt"
```

The mutation output matters more than the test results. Passing tests show the suite runs; the mutation log shows each rule was broken on purpose and the suite noticed.

Portfolio submission checklist

- [ ] `policies/` with five policies, each carrying a `# METADATA` block.
- [ ] `policies/tests/` with passing and failing fixtures for each.
- [ ] `scripts/mutation-test.sh` committed and executable.
- [ ] `evidence/lab-3-3/opa-test-results.json` from `opa test --format=json policies/`.
- [ ] `evidence/lab-3-3/mutation-results.txt`.
- [ ] `evidence/lab-3-3/policy-catalog.json`.
- [ ] `policies/README.md` listing each policy, control, severity, and remediation.

Troubleshooting

- **`rego_parse_error: yaml: mapping values are not allowed`** in METADATA. The YAML parser rejects unquoted colons inside a value. Quote `description` and `remediation`.
- **A passing fixture surprises you with a deny.** Module-wrapped resources live under `child_modules[]`, not `root_module.resources`. Use the shared `buckets` / `all_resources` helpers.
- **`encryption: [{}]` on a CMEK bucket.** Plan-time unknowns are omitted from the JSON. Require the block, not the value.
- **Set subtraction returns the wrong type.** Both operands must be sets. Use `{k | ...}`, not `[k | ...]`.

- **Mutation test reports SURVIVED on a rule you are sure works.** The `sed` mutation only touches `==`, so rules built purely from `count()` or `not` need a hand-written mutation. Add one, or accept that this rule needs a targeted test instead.
- `opa inspect` **shows no annotations.** The `# METADATA` block must sit immediately above the `package` declaration or a rule, with no blank line between.

Cleanup

Plan-only. Nothing deployed unless you applied the fixture, which the lab does not require. If you did:

```
cd terraform && terraform destroy -auto-approve
```

How this feeds the capstone

- **Ch 3.4** adds AWS variants targeting `aws_s3_bucket` and friends, and runs the combined suite through Conftest.
- **Ch 4.3** calls that gate on every PR.
- **Ch 6.1** reads `policy-catalog.json` to generate OSCAL `implemented-requirement` entries. Authored once, consumed twice.
- **The capstone** requires five or more Rego policies with tests, each citing a control from your declared framework. You have five, tested, mutation-checked, and self-describing. The remaining work is retargeting the control IDs from NIST 800-53 to HIPAA, SOC 2, or CMMC, which is a metadata edit rather than a rewrite. That portability is the entire argument for putting the control ID in the annotation instead of the filename.

Revision history

v2 (current)

- Two policies added: AU-3 (access logging configured) and AU-9 (the logging target is itself versioned).
- Added `scripts/mutation-test.sh`. A policy no test constrains cannot fail, so each one is broken on purpose and the tests must notice.
- Refactored the `child_modules` traversal into a shared helper rather than duplicating the `deny` rule per policy.
- `# METADATA` annotations are now consumed rather than decorative: Lab 6.1 extracts them with `opa inspect` to generate the OSCAL component.

v1

Initial release: three policies (SC-28, AC-3, CM-6), metadata blocks present but unused, traversal duplicated per rule.